

진행 보고: 2단 구조의 속도·정확도 교환 재측정과 넘김 판단 비교

학부 졸업논문 · 보고일 2026-10-04 · 직전 보고(2026-09-15) 이후분

1. 이번 주에 한 일

지난 보고 이후 논문 제목과 범위를 정리했고, 새 데이터셋(SR-BH 2020)에서 2단 구조를 처음으로 끝까지 측정했습니다. 실험은 네 가지입니다. 학습 입력에 쿠키를 넣을지 정하는 절제 실험, 1차 모델의 확률이 1.0으로 뭉치는지 확인한 점검, 넘김 비율과 실행 환경을 바꿔 가며 잔 속도·정확도 교환, 그리고 넘김 판단에 쓰는 확신도 신호 네 종류의 비교입니다. 판정 기준은 측정 전에 연구 기록에 먼저 적어 두었고, 아래 결과는 모두 그 기준대로 판정했습니다.

결과는 기대와 반대 방향으로 나왔습니다. 2기 데이터(GPU·배치 512)에서 5.07배였던 속도 이득이 새 데이터의 헤드라인 조건(CPU·배치 1)에서는 1.22배로 줄었고(같은 새 데이터에서 가장 유리한 조건도 1.79배), 실제 서비스처럼 공격이 드문 조건에서는 문자 단위 CNN 하나만 쓰는 것과 차이가 없거나 약간 느렸습니다. 확신도 신호를 바꾸는 방법으로는 이 문제가 풀리지 않는다는 것도 확인했습니다. 다만 두 결과 모두 원인이 숫자로 드러나서, 다음에 무엇을 해야 하는지는 오히려 분명해졌다고 생각합니다.

본문에서 쓰는 말을 먼저 정리합니다. 2단 구조는 가벼운 이미지 CNN(1차)이 모든 요청을 먼저 판정하고, 확신이 낮은 요청만 문자 단위 CNN(2차)에 넘기는 구성입니다. 넘김 비율은 전체 요청 중 2차로 넘어간 비율이고, 넘길지 말지는 1차가 낸 클래스 확률 중 가장 큰 값(최대 확률)이 임계값보다 낮은지로 정합니다. 오라클은 정답을 안다고 가정하고 "1차가 틀리고 2차가 맞는 요청만" 넘기는 이상적인 기준으로, 넘김 판단을 얼마나 더 잘할 수 있는지의 상한으로 씁니다.

2. 논문 제목과 범위

제목을 「표현 이질적 캐스케이드 웹 공격 탐지: 넘김 판단의 한계와 개선, 배포 조건별 속도-정확도 교환의 재평가」로 바꿨습니다. 지난달 정한 두 연구 축(넘김 판단, 속도·정확도 교환)을 제목에 그대로 드러낸 것입니다.

네 번째 연구 질문(암호화 경계에서 내용 기반 탐지가 어디서 무너지는가)은 이번 논문에서 뺐습니다. 이 질문은 7월 미팅에서 신규성이 약하다는 지적을 받고 추가했던 것인데, 2단 구조의 두 축과 연결되지 않아 새 제목 아래에 들어갈 자리가 없었습니다. 신규성은 이 질문 대신 두 축에서 보이려 합니다. 하나는 이미지 표현과 문자열 표현을 잇는 보안 탐지기에서 확신도 기반 넘김이 실제로 얼마나 부족한지 재는 것이고, 다른 하나는 같은 구조의 이득이 데이터와 배포 조건에 따라 얼마나 달라지는지 보이는 것입니다. 2기에 측정한 흐름 데이터 결과는 기록으로 남겨 두었고, 질문 번호도 다른 번호를 당기지 않고 비워 둡니다.

3. 학습 입력에서 쿠키를 뺀지

지난 보고에서 이미지 CNN은 쿠키를 입력에 넣었을 때 Macro-F1이 0.895에서 0.932로 올랐지만, 문자 단위 CNN은 거의 변하지 않았습니다. 명령 주입 요청에만 쿠키가 거의 항상 붙어 있어서(99.4%, 다른 클래스는 약 80%), 모델이 쿠키의 내용이 아니라 쿠키가 붙어 있다는 사실이나 길이를 단서로 썼을 가능성이 있었습니다. 이를 가르려고 쿠키 값을 같은 길이의 x 문자로 덮은 입력을 만들어 같은 조건으로 학습했습니다. 쿠키의 유무와 길이는 남고 내용만 사라지는 입력입니다.

입력	fold 별 Macro-F1	평균 ± 표준편차	오탐 1%일 때 탐지율
URI + 본문	0.885 · 0.893 · 0.911 · 0.878 · 0.907	0.895 ± 0.014	0.867
URI + 본문 + 쿠키	0.907 · 0.949 · 0.929 · 0.927 · 0.950	0.932 ± 0.018	0.850
URI + 본문 + 쿠키(내용 지움)	0.862 · 0.907 · 0.933 · 0.880 · 0.950	0.906 ± 0.036	0.866

이미지 CNN, 시험 데이터를 뺀 학습+검증 226,048건의 5겹 교차검증. 대응표본 t-검정, 미리 정한 두 쌍에 Holm 보정.

쿠키 원본과 내용을 지운 입력의 차이는 2.6%p 였지만 보정 p 가 0.17 로 유의하지 않았고, 내용을 지운 입력과 쿠키 없는 입력의 차이(1.2%p, p 0.35)도 마찬가지였습니다. 미리 정한 규칙에 따라 학습 입력은 URI + 본문으로 유지합니다. 정확히 말하면 "쿠키 내용이 쓸모없다"를 보인 게 아니라 "쿠키 내용이 도움이 된다"를 보이지 못한 것입니다. 다섯 fold 중 세 곳에서는 원본 쿠키 쪽이 4%p 넘게 높았으니, 내용을 지운 입력의 fold 간 편차가 커서 5겹으로는 가르기 어려웠다고 보는 편이 맞습니다. 그래서 9월에 내렸던 "쿠키 포함 입력 채택"은 철회가 아니라 보류로 기록했고, 이후 실험은 모두 URI + 본문 입력으로 진행했습니다.

4. 1차 모델 확률 포화 점검

넘김 판단은 1차의 최대 확률 순서에 기대기 때문에, 확률이 float32 에서 정확히 1.0 으로 뭉치면 그 덩어리 안에서는 순서를 매길 수 없습니다. 검증 데이터에서 이런 요청이 5%를 넘으면 확률 대신 로짓을 저장하도록 코드를 바꾸기로 했었는데, 검증·시험 모두 0%였고 서로 다른 최대 확률 값이 약 37,600 개 나왔습니다. 확률 그대로 진행했습니다.

5. 넘김 비율에 따른 정확도

넘김 비율을 0%에서 100%까지 정해 두고, 각 비율에 맞는 임계값을 검증 데이터에서 구한 뒤 시험 데이터에 한 번만 적용했습니다. 양 끝은 각각 이미지 CNN 단독(0%), 문자 단위 CNN 단독(100%)과 같습니다.

넘김 비율(목표)	시험 넘김 비율	Macro-F1	오탐 1%일 때 탐지율	정상 요청 오탐률	실사용자 요청 오탐
0% (이미지 CNN 단독)	0.0%	0.812	0.679	9.38%	8 / 95
5%	5.3%	0.853	0.730	6.84%	7 / 95
10%	10.2%	0.877	0.776	4.94%	6 / 95
20%	20.1%	0.915	0.868	2.72%	6 / 95
30%	30.0%	0.946	0.944	1.65%	5 / 95
50% (대표 운영점)	50.3%	0.981	0.992	0.94%	5 / 95
100% (문자 단위 CNN 단독)	100%	0.985	0.996	0.78%	5 / 95
오라클	15.3%	0.989	—	—	—

시험 데이터 약 4만 건(이 중 정상 10,042건, 실사용자 브라우저에서만 온 정상 요청 95건). 대표 운영점은 "검증 Macro-F1 이 문자 단위 CNN 단독보다 0.5%p 넘게 낮지 않은 가장 작은 넘김 비율"로 미리 정했습니다. 1%-2% 행은 생략했습니다.

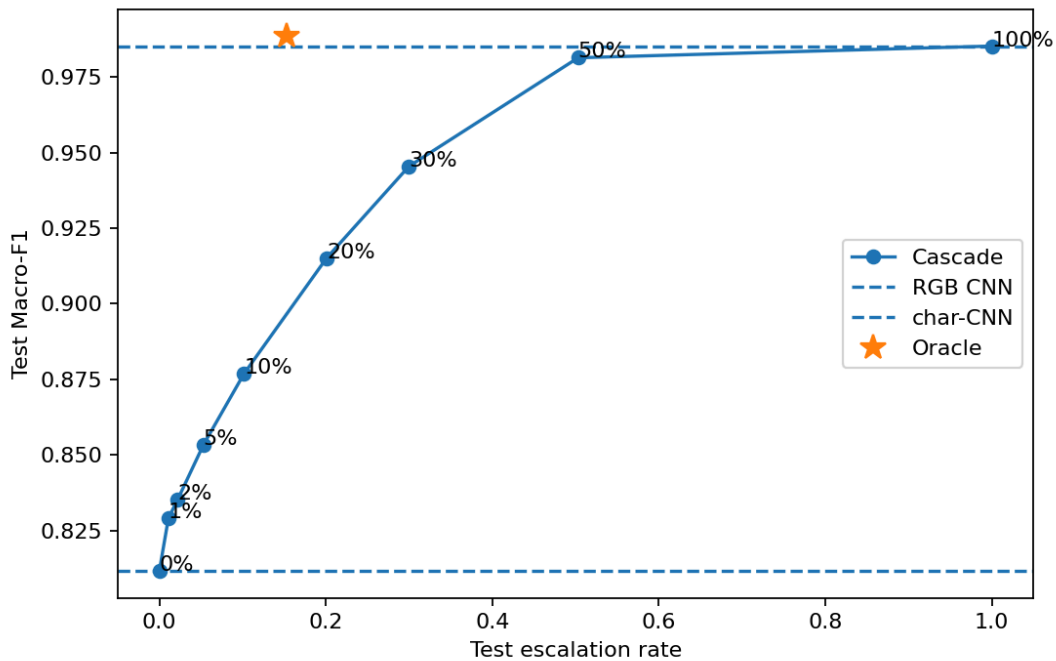


그림 1. 넘김 비율과 시험 Macro-F1. 아래 점선은 이미지 CNN 단독, 위 점선은 문자 단위 CNN 단독이고 별표는 오라클입니다. 두 점선이 같은 색으로 그려진 것은 그림 코드의 표시 문제로 수치와는 관계없습니다.

2기 데이터에서는 8%만 넘겨도 문자 단위 CNN 과 0.2%p 차이였는데, 이번에는 절반을 넘겨야 그 수준에 닿았습니다. 30%를 넘겨도 Macro-F1 이 0.946 에 머물렀습니다. 1차가 약한 것이 이유입니다. 이미지 CNN 단독 Macro-F1 이 0.81 이고 정상 요청의 9.4%를 공격으로 판정하는데, 2차와의 격차가 17%p 나 되니 넘겨야 할 요청이 많을 수밖에 없습니다.

클래스별로 보면 더 분명합니다. 대표 운영점에서 정상 요청은 70.7%가 2차로 넘어갔고, 공격 요청은 43.4%가 넘어갔습니다(명령 주입 83.7%, SQL 주입 43.8%, 코드 주입 13.7%). 1차가 공격보다 정상 요청을 덜 확신한다는 뜻이고, 이 점이 다음 절의 속도 결과를 좌우합니다. 오라클은 15.3%만 넘기고도 문자 단위 CNN 보다 약간 높으므로, 넘김 판단만 잘하면 줄일 여지는 큼니다.

실사용자 요청에 대한 오탐은 대표 운영점에서 문자 단위 CNN 단독과 같은 5건(95건 중)입니다. 95건으로는 신뢰구간이 2.3%~11.7%로 넓어서 두 구성의 차이를 말할 수는 없고, 2단 구조가 실사용자 오탐을 늘리지 않았다는 정도까지만 말할 수 있습니다.

6. 실행 환경별 지연

요청 하나를 처리하는 데 걸리는 시간을 장치(CPU, GPU)와 배치 크기(1, 32, 128, 1024)를 바꿔 가며 잴습니다. 헤드라인 조건은 요청이 하나씩 들어오는 웹 방화벽을 생각해 측정 전에 CPU배치 1로 정해 두었습니다(MLPerf의 single-stream 시나리오와 같은 설정). 2단 구조의 지연은 1차 지연에 "넘김 비율 × 2차 지연"을 더해 계산했습니다.

시험 데이터는 공격이 75%를 차지하는데 실제 서비스는 그 반대입니다. 그래서 공격 요청과 정상 요청의 넘김 비율(43.4%, 70.7%)을 따로 구한 뒤, 공격 비율이 3.3%와 0.1%일 때의 넘김 비율을 다시 계산해 함께 적었습니다. 공격이 드물수록 정상 요청의 넘김 비율이 전체를 결정하므로 넘김 비율은 약 70%로 올라갑니다.

장치 · 배치	이미지 CNN	문자 단위 CNN	2단 구조	속도 배수 (공격 74.8%)	속도 배수 (공격 3.3%)	속도 배수 (공격 0.1%)
CPU · 1 (헤드라인)	0.350	1.114	0.911	1.22	0.99	0.98
CPU · 32	0.132	0.901	0.585	1.54	1.18	1.17
CPU · 128	0.125	1.024	0.640	1.60	1.22	1.21
CPU · 1024	0.165	1.725	1.033	1.67	1.26	1.25
GPU · 1	0.209	0.235	0.327	0.72	0.63	0.63
GPU · 32	0.0081	0.0374	0.0269	1.39	1.09	1.08
GPU · 128	0.0040	0.0699	0.0392	1.79	1.32	1.31
GPU · 1024	0.0080	0.0634	0.0399	1.59	1.21	1.20

단위는 요청 1건당 ms(순전파만, 전처리 제외). 속도 배수 = 문자 단위 CNN 지연 ÷ 2단 구조 지연. 넘김 비율은 대표 운영점(50.3%) 기준이며, 배치가 1보다 크면 이 계산은 근사입니다. 측정 장비는 RTX 4080 SUPER, CPU 8 스레드. 각 조건은 한 번씩 측정했습니다.

헤드라인 조건의 속도 배수는 1.22배이고, 공격 비율을 실제 서비스 수준으로 맞추면 0.99배로 문자 단위 CNN 하나만 쓰는 것과 차이가 없습니다. GPU·배치 1에서는 두 모델의 지연이 거의 같아 2단 구조가 오히려 느립니다. 가장 유리한 조건(GPU·배치 128, 시험 데이터 공격 비율)에서도 1.79배였습니다. 측정 전에 정한 규칙이 "헤드라인 배수가 2배 미만이면 첫 번째 기여 주장을 조건부 이득으로 낮춘다"였으므로, 이번 결과로 그 주장을 하향했습니다.

두 가지 이유가 겹쳤습니다. 첫째, 두 모델의 비용 차가 줄었습니다. 넘김이 0%여도 CPU·배치 1에서 낼 수 있는 최대 배수는 3.18배인데, 2기에서는 처리량 기준으로 13.6배 차이가 났습니다. 둘째, 앞 절에서 본 것처럼 1차가 약해 절반 이상을 넘겨야 합니다. 2기의 5.07배는 2기 데이터의 값으로 그대로 두고, 논문에는 이번 결과를 조건과 함께 나란히 싣겠습니다.

7. 넘김 판단 신호 비교

넘김 판단에 최대 확률 대신 다른 신호를 쓰면 넘김을 줄일 수 있는지 봤습니다. 비교한 신호는 온도 보정한 최대 확률, 1·2위 확률 차(margin), 엔트로피입니다. 시험 데이터를 건드리지 않으려고 교차검증 5개 fold 각 각에서 두 모델의 확률을 저장해 뒀고, 온도는 fold 의 검증 부분에서, 평가는 fold 의 시험 부분에서 구했습니다. 지표는 "문자 단위 CNN 단독과 같은 Macro-F1 을 내는 데 필요한 넘김 비율 ÷ 오라클 넘김 비율"(과잉 넘김 배수)이고, 20% 이상 줄면서 보정 p 가 0.05 미만이면 개선으로 인정하기로 미리 정했습니다.

fold	오라클 넘김	최대 확률로 필요한 넘김	최대 확률	온도 보정	margin	엔트로피	온도 T
1	8.0%	60.5%	7.58	7.58	7.65	7.58	1.02
2	6.3%	82.5%	13.12	12.96	12.96	12.96	1.05
3	6.0%	48.5%	8.15	8.07	8.15	8.07	1.06
4	8.3%	50.5%	6.09	6.09	6.09	6.09	0.99
5	5.6%	99.0%	17.73	15.49	15.58	15.49	0.99
평균			10.53	10.04	10.09	10.04	

수치는 과잉 넘김 배수(작을수록 좋음). 최대 확률 대비 감소율은 온도 보정 4.7%, margin 4.3%, 엔트로피 4.7%이고, Holm 보정 p 는 세 쌍 모두 0.96 입니다. 필요한 넘김 비율은 0.5% 간격으로 찾았기 때문에 신호끼리 같은 값이 자주 나옵니다.

세 신호 모두 줄인 폭이 5% 남짓이고 통계적으로도 유의하지 않아, 미리 세운 가설은 기각입니다. 개선이 보인 곳도 fold 5 하나뿐이었습니다. 원인은 신호들이 사실상 같은 것을 재고 있다는 데 있습니다. 네 신호로 매긴 요청 순위의 상관이 fold 마다 0.997 이상이었고, 온도가 0.99~1.06 으로 1 에 붙어 있어 온도 보정을 해도 순서가 거의 바뀌지 않습니다. 어떤 신호를 쓰든 "1차가 덜 확신하는 요청"이라는 같은 집합을 고르는데, 그 집합이 오라클이 고르는 "1차가 틀리고 2차가 맞는 요청"과 다르다는 것이 평균 10.5배 과잉의 정체입니다.

2기에서詹 14.8배와 같은 지표가 새 데이터에서도 6.1~17.7배로 나왔으므로, 확신도 기반 넘김의 한계는 데이터가 바뀌어도 유지된다고 볼 수 있습니다. 다만 2기 값은 단일 분할에서 다른 목표 지표로詹 것이라 같은 자릿수라는 정도로만 비교하겠습니다. 이 결과는 아래 정리한 Jitkrittum 등(2023)의 분석, 즉 1차 확신도만 보는 규칙은 2차가 맞을지를 알 수 없다는 결과와 일치합니다. 신호를 바꾸는 방향은 여기서 접고, 넘김 판단 개선은 9절의 방법으로 접근하겠습니다.

8. 관련 논문 정리

연구 초점을 넘김 판단과 배포 지연으로 좁히고 보니, 지금까지 모은 문헌은 보안 응용과 조기종료 쪽뿐이고 넘김 규칙 자체를 다룬 기계학습 논문이 없었습니다. 이번 주에 13편을 새로 받아 원문 첫 쪽의 제목·저자를 대조해 확인했습니다. 학회명은 arXiv 메타데이터나 학회 페이지에서 확인했고, 확인하지 못한 한 편 (FrugalGPT)은 arXiv 로만 적었습니다.

논문	이 연구에서 쓰는 부분
Jitkrittum et al., When Does Confidence-Based Cascade Deferral Suffice? NeurIPS 2023	최적 넘김 규칙은 "2차가 1차보다 맞을 가능성이 더 높을가"로 정해지며, 1차 확신도만으로는 이를 알 수 없다는 분석. 7절 결과의 해석 틀이고, 학습형 넘김 규칙의 목표값(2차 정답 여부 - 1차 정답 여부)도 여기서 가져옴
Gupta et al., Language Model Cascades: Token-level Uncertainty and Beyond. ICLR 2024	분류 문제의 캐스케이드에서는 확신도 기반 넘김이 기본값이라는 정리. 관련연구에서 분류와 생성을 구분할 때 인용
Wang et al., Wisdom of Committees. ICLR 2022	학습 없이 확신도 임계값으로 기존 모델을 잇기만 해도 같은 정확도에서 2.3~5.4배 빨라진다는 결과. 기존 모델과 데이터가 달라 우리 배수와 직접 비교하지 않고 맥락으로만 씀
Kag et al., Efficient Edge Inference by Selective Query. ICLR 2023	엣지 모델과 라우터를 함께 학습해 서버가 맞힐 예제만 보내는 방법. 라우터 학습 목표와, 지연 조건별로 결과를 보고하는 방식의 참고
Enomoto & Eda, Learning to Cascade. AAAI 2021	기존 확률 보정을 캐스케이드에 그대로 쓰면 오히려 나빠질 수 있다는 보고. 온도 보정이 넘김을 줄이지 못한 7절 결과와 같은 방향
Hendrycks & Gimpel, A Baseline for Detecting Misclassified and Out-of-Distribution Examples. ICLR 2017	최대 확률을 오분류 탐지의 기준선으로 쓰는 근거. 현재 넘김 신호의 출처
Geifman & El-Yaniv, Selective Classification for Deep Neural Networks. NIPS 2017	판정 보류와 위험-커버리지 교환. 넘김 비율 곡선을 "1차의 판정 보류"로 읽는 개념적 근거
Madras et al., Predict Responsibly: Learning to Defer. NeurIPS 2018	뒤에 오는 판정자를 고려한 넘김 학습. 학습형 넘김 규칙의 이론 배경
Mozannar & Sontag, Consistent Estimators for Learning to Defer to an Expert. ICML 2020	넘김 학습을 비용 민감 학습으로 바꾼 일관된 손실. 우리는 학습이 끝난 모델 뒤에 붙이는 라우터 손실 자체는 쓰지 않고 배경으로만 인용
Chen et al., FrugalGPT. arXiv:2305.05176, 2023	대형 언어모델 캐스케이드에서 학습한 점수로 넘김을 정해 비용을 크게 줄인 사례. 다른 분야의 비용 중심 서술 사례
Teerapittayanon et al., BranchyNet. ICPR 2016	조기종료의 원형. 2기에 시도했다가 채택하지 않은 조기종료 1차의 인용
Reddi et al., MLPerf Inference Benchmark. ISCA 2020	추론 성능이 시나리오(요청 1건씩, 다중 스트림, 서버, 오프라인)에 따라 크게 달라진다는 점. 헤드라인을 CPU·배치 1 로 고정할 근거
Ovadia et al., Can You Trust Your Model's Uncertainty? NeurIPS 2019	분포가 바뀌면 사후 보정만으로는 불확실성이 맞지 않는다는 결과. 다른 데이터셋으로 임계값을 옮기는 다음 실험의 예상 근거

이 묶음을 읽고 정리한 점은 세 가지입니다. 첫째, "확신도 넘김이 언제 충분한가"는 일반 기계학습에서 이미 다뤄진 질문이라 우리 기여를 넘김 규칙 개선 전반으로 넓게 쓰면 안 됩니다. 표현이 다른 두 웹 공격 탐지기를 이은 구조에서 그 조건이 성립하는지 실측하는 것으로 좁혀 쓰겠습니다. 7절 결과가 바로 그 실측입니다. 둘째, 학습형 넘김 규칙의 목표값은 Jitkrittum 등의 정의를 그대로 쓸 수 있어 새로 설계할 부분이 적습니다. 셋째, 속도 배수는 기존 모델·장치·배치·공격 비율을 붙여서만 쓰겠습니다. 다른 논문의 배수는 조건이 달라서 나란히 적더라도 직접 비교하지 않겠습니다.

9. 해석과 다음 계획

이번 결과로 2단 구조의 이득이 "1·2차의 비용 차"와 "얼마나 적게 넘기는가"의 곱이라는 점이 분명해졌고, 새 데이터에서는 두 항이 모두 나빠졌습니다. 신호를 바꾸는 쉬운 방법은 효과가 없었으니, 남은 방법은 넘길 요청을 더 잘 고르는 것과 1차 자체를 강하게 만드는 것입니다. 다음 주에는 아래 순서로 진행하려 합니다.

1. 클래스별 임계값. 넘김의 다수가 정상 요청이고, 실제 서비스에서는 정상 요청의 넘김 비율이 곧 전체 넘김 비율입니다. 1차가 정상으로 판정한 요청에 별도 임계값을 두는 것만으로 줄일 여지가 있는지 먼저 보

니다. 추가 학습이 없어 비용이 가장 작습니다.

2. 학습형 넘김 규칙. 검증 데이터를 다시 나눠 "2차가 1차보다 맞을지"를 예측하는 작은 모델을 1차 출력 위에 학습합니다. 오라클과 실제 사이 간격이 10배라서, 개선이 있다면 여기서 나와야 합니다.
3. 1차 강화. 1차와 2차의 Macro-F1 격차(17%p)가 넘김 50%의 근본 원인입니다. 1차를 조금 키워 1차 지연이 늘더라도 넘김이 크게 줄면 전체로는 빨라질 수 있어서, 크기를 몇 단계로 바꿔 교환 곡선을 다시 그려볼 계획입니다.

상의드리고 싶은 점이 하나 있습니다. 첫 번째 기여 주장을 조건부로 낮추면서 논문의 무게가 "2단 구조가 빠르다"에서 "언제 빠르고 언제 아닌지, 그 차이가 넘김 판단에서 온다"로 옮겨 갑니다. 제목의 "한계와 개선, 재평가"와는 맞는 방향이지만, 이번 결과를 부정 결과로 정면에 돌지 개선 실험의 출발점으로 돌지에 따라 장 구성이 달라집니다. 다음 미팅 때 의견을 여쭙고 싶습니다.

참고: 재현 정보

모든 수치는 저장소의 연구 기록(docs/14 §8.2~§8.4)과 실험 결과 파일에서 가져왔습니다. 교차검증 확률은 다시 학습해서 저장했는데, 이미지 CNN 은 9월 결과와 다섯 fold 모두 소수 넷째 자리까지 같았고 문자 단위 CNN 은 fold 별로 최대 0.8%p 차이가 났습니다(평균 0.988 대 0.990, GPU 연산의 비결정성). 7절의 목표값은 이번에 저장한 확률에서 다시 계산했으므로 판정에는 영향이 없습니다. 5.6절의 확률과 임계값은 추가 학습 없이 기존 체크포인트에서 계산했습니다.